COMP438/538 Reinforcement Learning

Bellman Equations and Dynamic Programming

Barış Akgün

Fall 2026

So Far

- Learning from Experience
- The Interaction Loop: state-action-reward trajectories
- Policies and (discounted) returns
- State and State-Action Values as expected returns

- Formalizing it all: MDPs

- Where we left off?
  - How to pick between two policies?
  - How to calculate expected returns?
  - What to do when the trajectories do not end?

- Reminder: Episodic vs Continuing Problems (and terminal/absorbing states)

Brainstorming

- Given a model of the environment and the policy, how can we calculate returns?

Evaluating a policy

Prediction (policy evaluation): calculate a (fixed) policy's state and state-action values.

- Averaging returns
  - Sample Variance
  - Tail for continuing trajectories
- Using a model
  - Handles continuing trajectories
  - Leverages the Markovian nature of the problem

Control: find a policy with higher expected return.

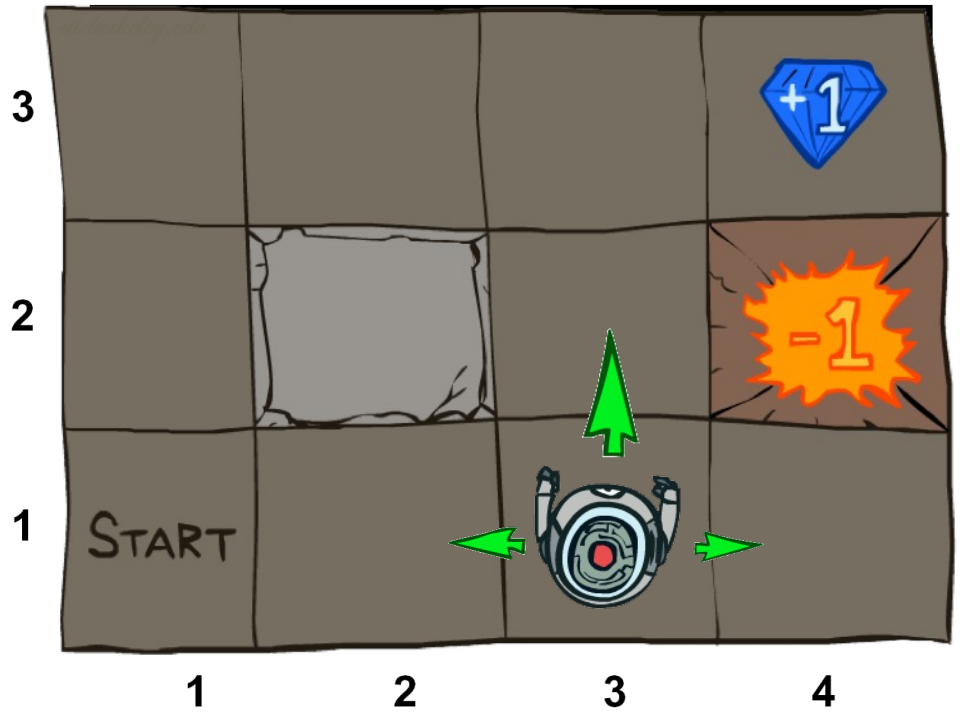
Values account for delayed consequences and help us compare decisions.

Gridworld as an MDP

Formulate this as an MDP:

What are the states, actions, model, rewards and discount?

- States: Cells on the grid
- Actions: up, down, left, right
- Model:
  - Intended direction: 0.8
  - each perpendicular direction: 0.1
  - A blocked movement leaves the agent in the same cell
- Entering either terminal cell ends the episode; no rewards follow
- $\gamma = 0.5$ for today's examples



(4,3): **+1 on entry** (4,2): **−1 on entry**

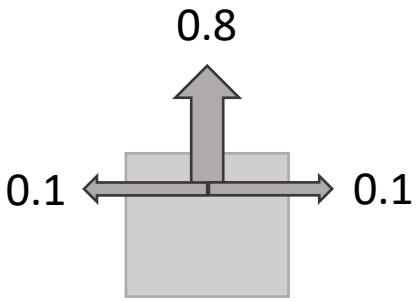
Other transitions: living reward −0.04

A smaller grid for calculations

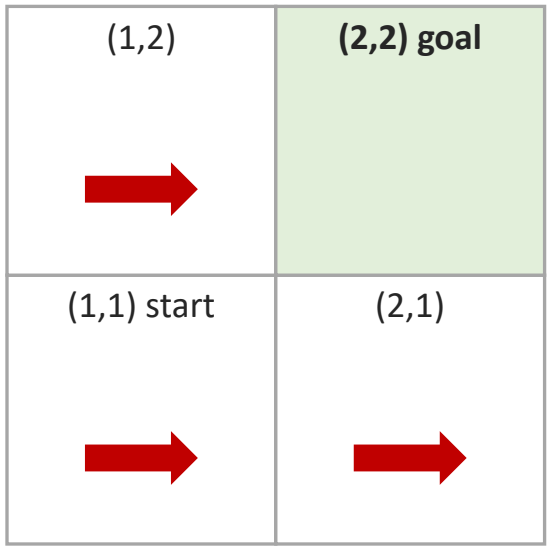
Same movement rules, smaller map. This is a separate MDP.

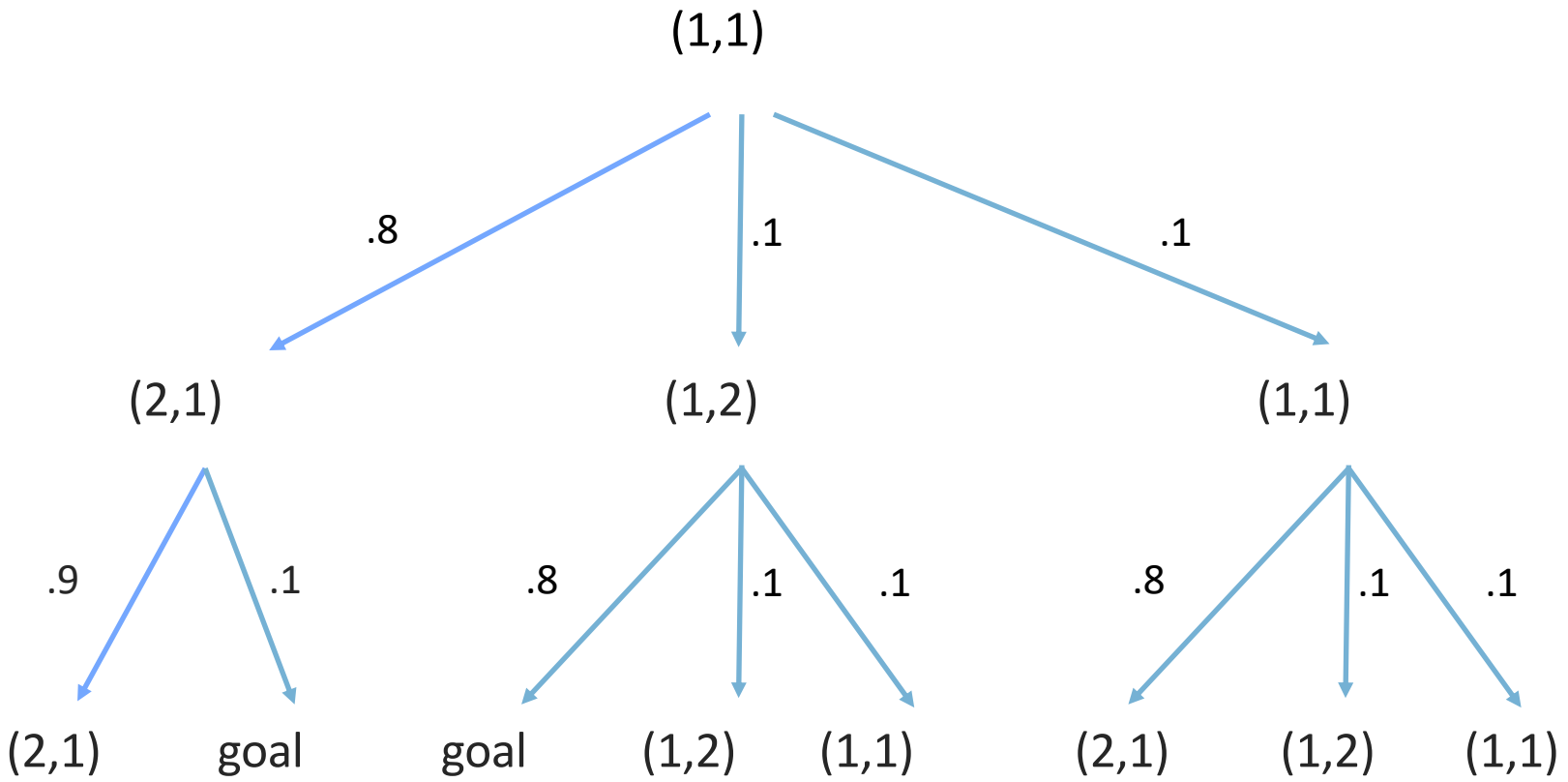
- Goal entry: **+1** total; every other transition before termination: **–0.04**
- Goal continuation value 0; discount $\gamma = 0.5$
- Fixed policy π: choose **right** at every nonterminal cell

(1,2)	**(2,2) goal**
(1,1) start	(2,1)



Two-step transition prediction





Under the always-right policy, what is the probability of reaching the goal in two steps?

Reusing future subproblems

- Different routes can arrive at the same state.
- From that state, the same policy faces the same future problem.
- Store one value per state and reuse it when evaluating predecessors.

Dynamic programming uses this repeated structure to compute values.

Back to Values

- State-Action Value definition: $q^{\pi}(s,a) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$
- Given a trajectory $\ldots S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, R_{t+2}, \ldots$, discounted return: $G_t = R_{t+1} + \gamma G_{t+1}$

$$\Rightarrow q^{\pi}(s,a) \doteq \mathbb{E}_{\pi}[\, R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a \,]$$
$$= \mathbb{E}[R_{t+1} | S_t = s, A_t = a] + \gamma \mathbb{E}_{\pi}[\, G_{t+1} \mid S_t = s, A_t = a \,]$$

- Recall that $\mathcal{R}(s,a) \doteq \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$
- With the Markov State Assumption:

$$\mathbb{E}_{\pi}[\, G_{t+1} \mid S_t = s, A_t = a \,] = \sum_{s' \in \mathcal{S}^+} \underbrace{\Pr\{S_{t+1} = s' | S_t = s, A_t = a\}}_{\mathcal{T}(s,a,s')} v^{\pi}(s')$$

State and State-Action Values

- Plug everything back:

$$q^{\pi}(s,a) = \mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^{+}} \mathcal{T}(s,a,s') v^{\pi}(s')$$

- Deterministic Policy ($a = \pi(s)$):

$$v^{\pi}(s) = q^{\pi}\big(s, \pi(s)\big)$$

- Stochastic Policy ($\pi(a \mid s) = \Pr\{A_t = a \mid S_t = s\}$):

$$v^{\pi}(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) q^{\pi}(s,a)$$

Check: if the agent acts differently after the first action, can that action's value change?

Bellman Expectation Equation

$$v^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$
$$= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$
$$= \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \sum_{s' \in \mathcal{S}^+, r} \mathcal{M}(s, a, r, s')[r + \gamma v^{\pi}(s')]$$

Equivalent form using expected reward and next-state probabilities:

$$v^{\pi}(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s)[\mathcal{R}(s, a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s, a, s') v^{\pi}(s')]$$

Expected immediate reward + discounted expected continuation.

Deterministic Policy :

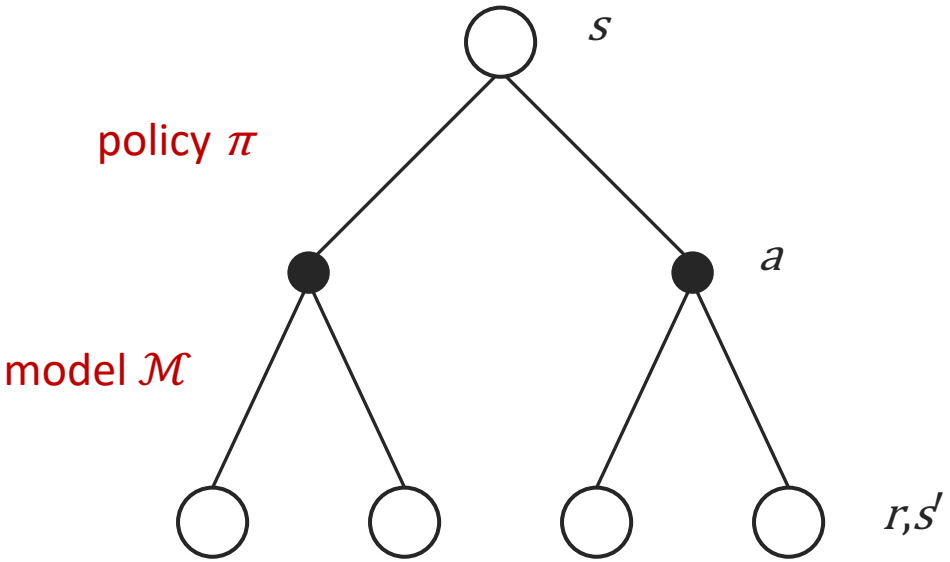
$$v^{\pi}(s) = \mathcal{R}(s, \pi(s)) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s, \pi(s), s') v^{\pi}(s')$$

Action-value recursion and backups

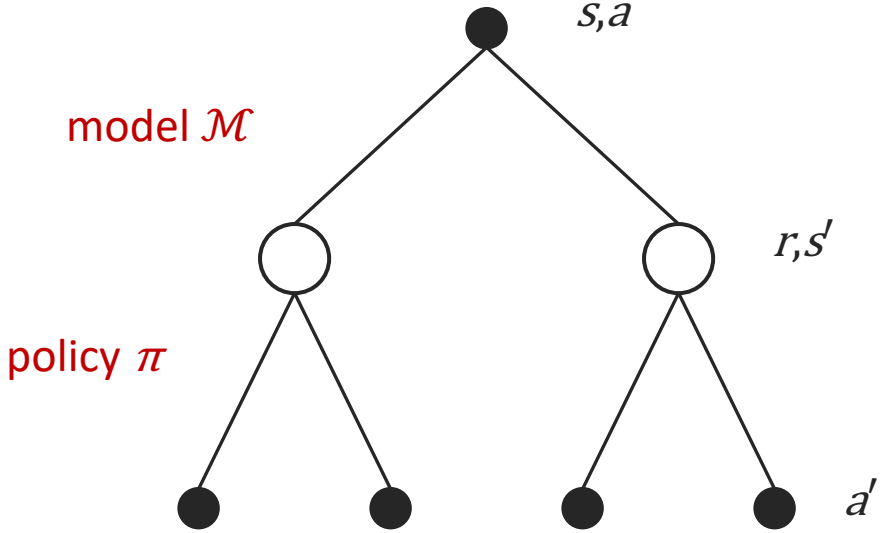
$$q^{\pi}(s,a) = \mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{T}(s,a,s') \sum_{a' \in \mathcal{A}(s')} \pi(a' \mid s') q^{\pi}(s',a')$$

A backup carries value information from possible successors to the state or action being evaluated.

State value $v^{\pi}(s)$

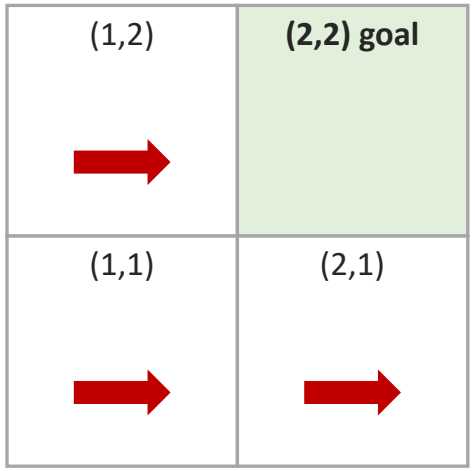


Action value $q^{\pi}(s,a)$



Three states, three value equations

- Policy π: always right. Goal continuation value 0.
- The paths may be arbitrarily long; this policy has only **three** unknown nonterminal values.



$$v^{\pi}((1,1)) = -0.04 + 0.5[0.1v^{\pi}((1,1)) + 0.1v^{\pi}((1,2)) + 0.8v^{\pi}((2,1))]$$

$$v^{\pi}((1,2)) = ?$$

$$v^{\pi}((2,1)) = ?$$

Solving the value equations

Fixed policy: a linear system for the unknown state values.

$$v^{\pi}((1,1)) = -0.04 + 0.5[0.1v^{\pi}((1,1)) + 0.1v^{\pi}((1,2)) + 0.8v^{\pi}((2,1))]$$

$$v^{\pi}((1,2)) = 0.792 + 0.5[0.1v^{\pi}((1,1)) + 0.1v^{\pi}((1,2))]$$

$$v^{\pi}((2,1)) = 0.064 + 0.5[0.9v^{\pi}((2,1))]$$

Optional Home Exercise: Write the system in matrix form and solve it.

Question: What is the complexity of this way of performing policy evaluation for an MDP with n states?

Iterative policy evaluation

$$V_{k+1}(s) \leftarrow \sum_{a \in \mathcal{A}(s)} \pi(a \mid s)[\mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') V_k(s')]$$

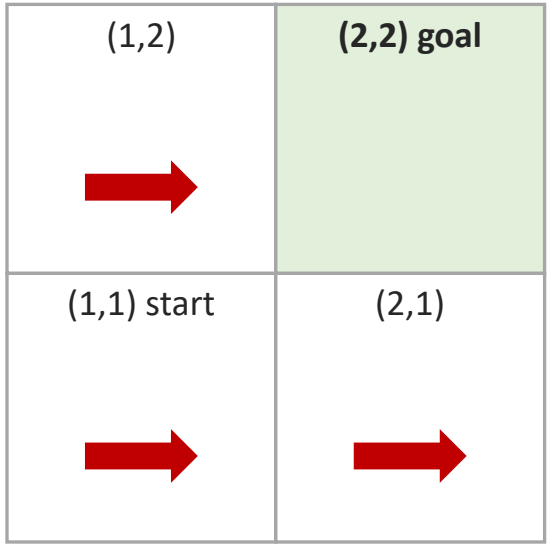
- Initialize $V_0(s) = 0$, including the terminal state.
- Compute every new entry using the **old values**.
- After all states are updated, replace the table and repeat.
- Stop when the largest change over a sweep is *sufficiently* small.

k **counts computational sweeps. The policy stays fixed.**

Complexity per step?

Total complexity?

First evaluation sweep

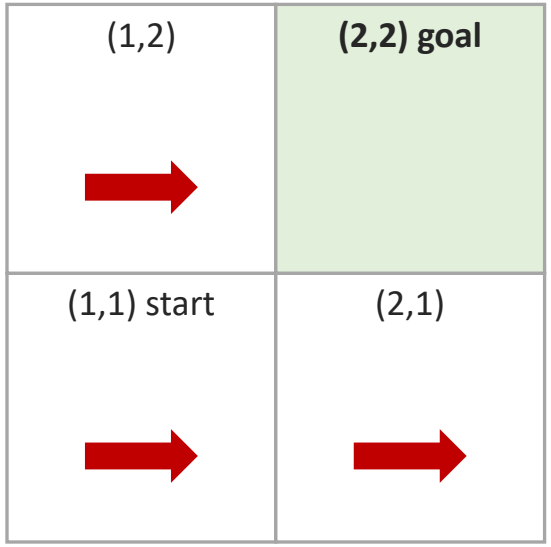


Sweep	(1,1)	(1,2)	(2,1)	Terminal
V_0	0	0	0	0
V_1	−0.04	0.792	0.064	0

With zero initial continuation values, the first sweep gives expected immediate rewards.

- Always-right policy, $\gamma = 0.5$, $V_0 = 0$.

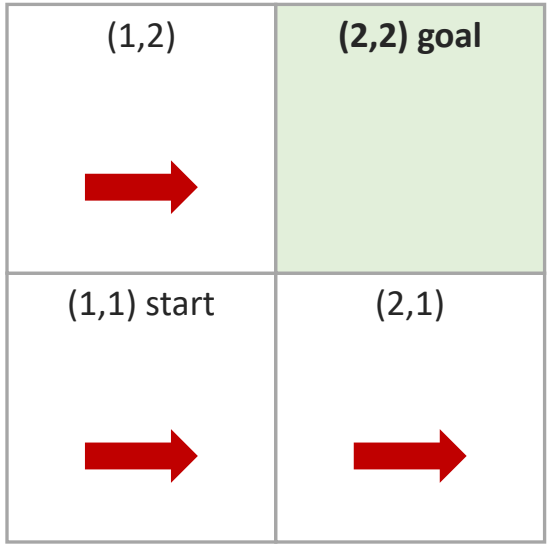
Second evaluation sweep



Sweep	(1,1)	(1,2)	(2,1)	Terminal
V_1	−0.04	0.792	0.064	0
V_2	?	?	?	0

Compute V_2 using V_1 for every successor value. Keep the policy fixed.

What the sweeps mean



Sweep	(1,1)	(1,2)	(2,1)	Terminal
V_1	−0.04	0.792	0.064	0
V_2	0.0232	0.8296	0.0928	0
V_3	0.03976	0.83464	0.10576	0

Starting from $V_0 = 0$, k synchronous sweeps give the exact expected discounted sum of the first k rewards under this policy.

Why repeated backups settle

Bellman (expectation) operator B^π: maps any value table V to a new table

$$(B^\pi V)(s) \doteq \sum_{a \in \mathcal{A}(s)} \pi(a \mid s)[\mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s')V(s')]$$

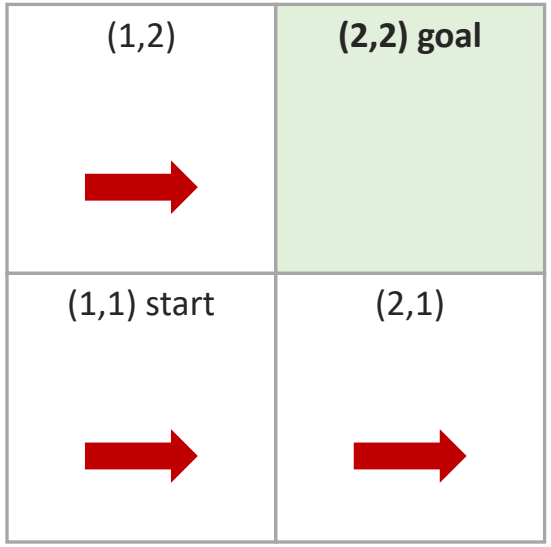
$$V_{k+1} = B^\pi V_k, \qquad B^\pi v^\pi = v^\pi$$

- B^π applies one full policy-evaluation sweep.
- Probability averaging cannot increase the largest discrepancy; discounting multiplies it by γ.

$$\max_{s \in \mathcal{S}^+} |V_{k+1}(s) - v^\pi(s)| \leq \gamma \max_{s \in \mathcal{S}^+} |V_k(s) - v^\pi(s)|$$

Finite MDP, bounded rewards, fixed policy, $0 \leq \gamma < 1$: the sweeps converge to its unique value function.

A better first action?



At (2,1), compare continuing right with choosing up once, then following the same policy.

$$v^{\pi}((2,1)) = q^{\pi}((2,1), \text{right}) = 32/275 \approx 0.11636$$

$$q^{\pi}((2,1), \text{up}) = 0.792 + 0.5[0.1(14/275) + 0.1(32/275)] \approx 0.80036$$

Question: How to formalize this step?

Optimal values

$$v^*(s) = \max_\pi v^\pi(s), \qquad q^*(s,a) = \max_\pi q^\pi(s,a)$$

- $v^*(s)$: best expected return from s.
- $q^*(s,a)$: expected return after first taking a, then acting optimally.

$$v^*(s) = \max_{a \in \mathcal{A}(s)} q^*(s,a)$$

Bellman optimality equations

$$v^*(s) = \max_{a \in \mathcal{A}(s)} [\mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}^+} \mathcal{T}(s,a,s') v^*(s')]$$

$$q^*(s,a) = \mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{T}(s,a,s') \max_{a' \in \mathcal{A}(s')} q^*(s',a')$$

The agent chooses an action; the environment's outcomes are still averaged.

Side Note: For finite discounted MDPs, there always exists an optimal deterministic policy.

Next: policy improvement, value iteration and policy iteration.